نام دانشجو: شروین آقائی

فعالیت کلاسی فصل ۴ — صفحه ۵۶ جزوه

کانتینرسازی سرویس Back-End با استفاده از Docker

Project: Smart Team Task Manager (Containerization & Deployment)

ارائه‌دهنده	موضوع فعالیت	خروجی مورد انتظار
شروین آقائی	Docker, Image & Container	Running Container + README

گام اول و دوم: ایجاد فایل‌های Dockerfile و .dockerignore

تعریف دستورالعمل ساخت Image استاندارد، سبک و امن برای سرویس Back-End پروژه

شروین آقائی

۱. محتوای فایل Dockerfile

بهره‌گیری از تصویر پایه سبک node:22-alpine و بهینه‌سازی کش لایه‌ها برای نصب سریع پکیج‌ها:

```
# 1. انتخاب Base Image رسمی و کم‌حجم
FROM node:22-alpine

# 2. تنظیم دایرکتوری کاری داخل کانتینر
WORKDIR /app

# 3. کپی فایل‌های وابستگی برای استفاده از Layer Cache
COPY package*.json ./

# 4. نصب پکیج‌های مورد نیاز محیط اجرا
RUN npm ci --only=production

# 5. کپی سورس‌کد اصلی پروژه
COPY . .

# 6. معرفی پورت پیش‌فرض سرویس Back-End
EXPOSE 3000

# 7. دستور اجرای برنامه هنگام شروع کانتینر
CMD ["npm", "start"]
```

۲. محتوای فایل .dockerignore

حذف فایل‌های اضافی، سنگین و محرمانه از Build Context:

```
# وابستگی‌های محلی و فایل‌های لاگ
node_modules
npm-debug.log*

# تاریخچه کنترل نسخه
.git
.gitignore

# فایل‌های محرمانه و متغیرهای محیطی
.env
.env.local

# فایل‌های داکر و مستندات غیرضروری در اجرا
Dockerfile
.dockerignore
README.md
```

گام سوم و چهارم: ساخت Image و اجرای Container با Port Mapping

اجرای دستورات Build، راه‌اندازی کانتینر در پس‌زمینه و تست ارتباط با API

شروین آقائی

دستورات ترمینال (Docker CLI Workflow)

```
# گام ۱: ساخت Docker Image با نام و تگ مشخص
docker build -t smart-task-api:1.0 .

# گام ۲: بررسی لیست Imageهای ساخته شده
docker images

# گام ۳: اجرای Container با نگاشت پورت 3000 هاست به 3000 کانتینر
docker run -d --name task-backend -p 3000:3000 smart-task-api:1.0

# گام ۴: مشاهده کانتینرهای در حال اجرا (Running Containers)
docker ps

# گام ۵: بررسی لاگ‌های خروجی سرویس
docker logs task-backend
```

خروجی اجرای کانتینر (Running Container)

```
$ docker ps
CONTAINER ID   IMAGE                COMMAND
STATUS         PORTS                    NAMES
a8f4c92d11e0   smart-task-api:1.0   "npm start"    Up 2
minutes   0.0.0.0:3000->3000/tcp   task-backend

$ docker logs task-backend
[INFO] Server initialized successfully.
[INFO] Smart Team Task Manager API listening on port 3000...

$ curl http://localhost:3000/api/tasks
{"status":"success","data":[{"id":"tsk_504","title":"طراحی
REST API"}]}
```

توضیح Port Mapping: پارامتر `-p 3000:3000` پورت ۳۰۰۰ سیستم میزبان (Host) را به پورت ۳۰۰۰ داخل کانتینر متصل می‌کند تا درخواست‌های HTTP به سرویس برسند.

خروجی مستندات: فایل راهنمای دستورات (README.md)

مستندسازی استاندارد برای ساخت، اجرا، مانیتورینگ و توقف کانتینر توسط اعضای تیم

شروین آقائی

محتوای فایل README.md — راهنمای اجرای کانتینری Back-End

```
# Smart Team Task Manager - Back-End API

 سرویس مدیریت وظایف تیمی مبتنی بر معماری Modular Monolith و REST
API.

## پیش‌نیازها (Prerequisites)
- نصب بودن Docker Engine (نسخه 24 به بالا)
- آزاد بودن پورت 3000 روی سیستم میزبان

## ۱. ساخت تصویر داکر (Build Image)
در مسیر اصلی پروژه دستور زیر را اجرا کنید:
```bash
docker build -t smart-task-api:1.0 .
```
```

```
## ۲. اجرای کانتینر (Run Container)
```bash
docker run -d --name task-backend -p 3000:3000 smart-task-api:1.0
```

۳. بررسی سلامت و مدیریت کانتینر
- **مشاهده کانتینرهای فعال:** `docker ps`
- **مشاهده زنده لاگ‌ها:** `docker logs -f task-backend`
- **توقف و حذف کانتینر:**
```bash
docker stop task-backend && docker rm task-backend
```
```

جمع‌بندی خروجی فعالیت فصل ۴ (Containerization)

مزایای فنی حاصل از کانتینرسازی سرویس Back-End پروژه Smart Team Task Manager

شروین آقائی

استانداردسازی و تکرارپذیری محیط اجرا

- **حذف مغایرت محیط‌ها:** برنامه همراه با نسخه دقیق Node.js و وابستگی‌ها بسته‌بندی شده و روی هر سرور یا سیستم شخصی به صورت یکسان اجرا می‌شود.
- **سرعت بالای راه‌اندازی:** به دلیل استفاده از هسته مشترک میزبان (Shared Kernel)، کانتینر در چند ثانیه و با حداقل مصرف RAM اجرا می‌شود.
- **آمادگی برای استقرار ابری (Cloud):**تولیدشده مستقیماً قابل آپلود در Image است. CI/CD و اجرا در سرورهای ابری و Registry.

بهینه‌سازی و نکات امنیتی اعمال‌شده

- **کاهش حجم Image:** استفاده از تصویر پایه `alpine` و نادیده گرفتن `node_modules` محلی باعث کاهش چشمگیر حجم نهایی Image شده است.
- **حفظ محرمانگی اطلاعات:** با قرار دادن `.env` و `.git` در فایل `.dockerignore`، از نشت کلیدهای امنیتی به داخل Image جلوگیری شده است.
- **مستندسازی کامل:** وجود فایل `README.md` امکان راه‌اندازی سریع پروژه را برای سایر اعضای تیم فراهم می‌کند.